



南開大學
Nankai University

人工智能技术实验 实验报告

实验名称：基于蚁群算法的旅行商问题

姓 名：殷家兴

学 号：2313666

专 业：自动化

人工智能学院

2026 年 1 月

一 问题简述

旅行商问题：设有 N 个互相可直达的城市，给定每对城市之间的距离，某推销商准备从其中的 A 城出发，周游各城市一遍，最后又回到 A 城，要求访问每一座城市并回到起始城市的最短回路。这是非常经典的组合优化问题中的 NP 难问题（多项式复杂程度的非确定性问题），通过常规的暴力枚举，遍历等方法难以计算出最优解，因此，本实验使用蚁群算法进行最优解的计算。

蚁群算法 (Ant Colony Optimization, ACO) 是一种基于群体智能的优化算法，由 Marco Dorigo 于 1992 年提出。蚁群算法模拟了自然界中蚂蚁群体寻找最短路径的行为，利用蚂蚁在行走过程中留下的信息素来引导后续蚂蚁选择路径。通过不断迭代和信息素更新，算法逐渐找到问题的最优解或近似最优解。

蚁群算法的核心思想是利用信息素 (Pheromone) 的正反馈机制来逐步优化解。蚂蚁在行走过程中会释放信息素，信息素的浓度与路径的质量（通常是路径的长度或花费的时间）有关。路径越短，信息素越浓，后续蚂蚁选择该路径的概率就越高。这种机制可以通过多次迭代不断强化优质路径，逐渐逼近最优解。

对于本实验，求解我国 34 个城市的旅行商问题，给定 34 个城市的名称和坐标数据，求遍历一次所有城市回到起点期间，所经过路程的较小值。

基本思想：

1. 根据具体问题设置多只蚂蚁，分头并行搜索
2. 每只蚂蚁完成一次周游后，在行进的路上释放信息素，信息素量与解的质量成正比。
3. 蚂蚁路径的选择根据信息素强度大小（初始信息素量设为相等），同时考虑两点之间的距离，采用随机的局部搜索策略。这使得距离较短的边，其上的信息素量较大，后来的蚂蚁选择该边的概率也较大。
4. 每只蚂蚁只能走合法路线（经过每个城市 1 次且仅 1 次）。
5. 所有蚂蚁都搜索完一次就是迭代一次，每迭代一次就对所有的边做一次信息素更新，然后新的蚂蚁进行新一轮搜索。
6. 更新信息素包括原有信息素的蒸发和经过的路径上信息素的增加。
7. 达到预定的迭代步数，或出现停滞现象（所有蚂蚁都选择同样的路径，解不再变化），则算法结束，以当前最优解作为问题的最优解。

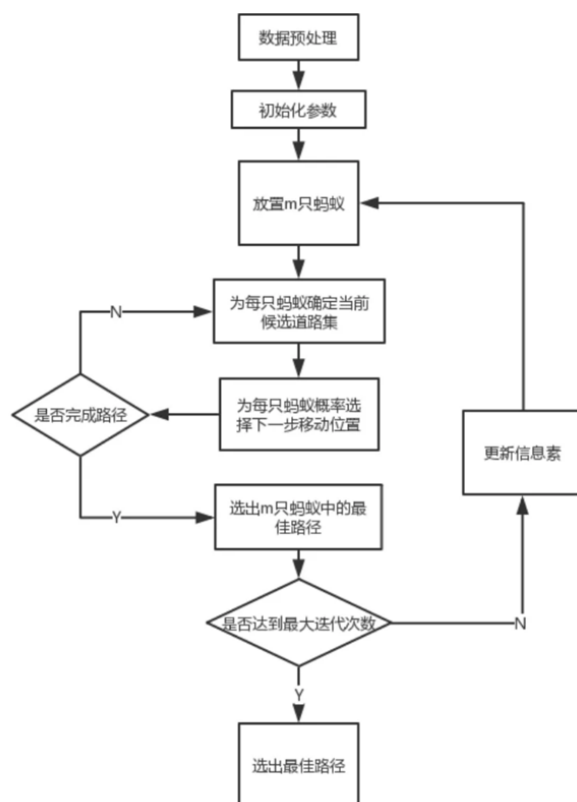


图 1: 算法实现流程

二 实验目的

1. 了解旅行商问题的基本概念，理解旅行商问题的求解难度；
2. 了解蚁群算法的基本原理及实现流程；
3. 能够利用蚁群算法解决旅行商问题；
4. 进一步理解可视化设计。（选做）

三 实验内容

实验包括自主起降、路径规划、遍历覆盖、编队协同四个子任务，每个子任务的实验内容如下：

1. 实现基于蚁群算法的 34 个城市的旅行商问题，求解访问每一座城市一次并最终回到起始城市的最短回路；
2. 对比蚁群算法下不同蚂蚁数量 m 、信息素挥发系数 ρ 、信息素总量 Q 、信息素因子 α 、启发函数因子 β 等参数下，遗传群算法对问题求解的效果影响并分析原因。

3. 进行蚁群算法可视化，展示搜索过程中每次迭代过程中种群最优路径走势的变化和迭代最优结果。（选做）

四 编译环境

- 操作系统：Windows 11;
- Python 版本：3.9.23;
- 依赖库：numpy、PIL (Pillow)、random、time、os、sys;
- 集成开发环境 (IDE)：VS Code。

五 实验步骤

5.1 城市数据加载与距离矩阵构建

程序内置中国 34 个主要城市的经纬度坐标，并通过计算两两点位之间的欧氏距离矩阵：

```
1 def _calc_distance_matrix(self):
2     diff = self.cities[:, np.newaxis, :] - self.cities[np.newaxis, :, :]
3     return np.sqrt(np.sum(diff**2, axis=2))
4     return np.sqrt(np.sum(diff ** 2, axis=2))
```

同时构建启发信息矩阵（距离的倒数），距离越短的城市对启发信息值越大：

```
1 self.heuristic_matrix[i, j] = 1.0 / self.distance_matrix[i, j]
```

5.2 算法参数初始化

初始化蚁群算法的关键参数：

- 蚂蚁数量 (m)：影响搜索能力，默认 50;
- 信息素重要性因子 (α)：控制信息素影响程度，默认 1.0;
- 启发信息重要性因子 (β)：控制距离启发影响程度，默认 3.0;
- 信息素挥发系数 (ρ)：控制信息素挥发速度，默认 0.3;
- 信息素总量 (Q)：控制信息素增量规模，默认 100.0;
- 最大迭代次数：控制算法运行时间，默认 200.

同时初始化信息素矩阵，初始所有边信息素浓度为 1。

5.3 蚂蚁路径构建（状态转移规则）

每只蚂蚁独立构建完整路径，采用轮盘赌选择法确定下一个访问城市，其概率计算公式如下：

$$P_{ij} = \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{k \in \text{allowed}} [\tau_{ik}]^\alpha \cdot [\eta_{ik}]^\beta} \quad (1)$$

其中：

- τ_{ij} ：城市 i 到 j 的信息素浓度
- η_{ij} ：启发信息， $\eta_{ij} = \frac{1}{d_{ij}}$
- α ：信息素重要因子
- β ：启发信息重要性因子

```
1 def _select_next_city(self, current_city, visited_cities):
2     allowed_cities = [city for city in range(self.num_cities)
3                       if city not in visited_cities]
4
5     # 计算概率
6     probabilities = []
7     for city in allowed_cities:
8         pheromone = self.pheromone_matrix[current_city, city] ** self.alpha
9         heuristic = self.heuristic_matrix[current_city, city] ** self.beta
10        probabilities.append(pheromone * heuristic)
11
12    # 轮盘赌选择
13    selected_idx = np.random.choice(range(len(allowed_cities)), p=
14    probabilities)
15    return allowed_cities[selected_idx]
```

5.4 信息素更新机制

信息素更新包含两部分：信息素挥发和信息素增量释放。

1. 信息素挥发：所有边上的信息素按比例挥发，防止信息素无限积累：

$$\tau_{ij}(t+1) = (1 - \rho) \cdot \tau_{ij}(t) \quad (2)$$

2. 信息素增量：每只蚂蚁根据其找到的路径质量释放信息素，路径越短释放越多：

$$\Delta\tau_{ij} = \frac{Q}{L_k} \quad (3)$$

其中 L_k 为蚂蚁 k 找到的路径长度。

```

1 def _update_pheromone(self, ant_paths, ant_distances):
2     # 信息素挥发
3     self.pheromone_matrix *= (1.0 - self.rho)
4
5     # 每只蚂蚁释放信息素
6     for ant_id in range(self.num_ants):
7         path = ant_paths[ant_id]
8         distance = ant_distances[ant_id]
9         delta_pheromone = self.Q / distance
10
11        # 在路径每条边上增加信息素
12        for i in range(len(path) - 1):
13            from_city = path[i]
14            to_city = path[i+1]
15            self.pheromone_matrix[from_city, to_city] += delta_pheromone
16            self.pheromone_matrix[to_city, from_city] += delta_pheromone

```

5.5 迭代优化与记录

主循环指定迭代次数，每只蚂蚁构建路径、计算距离、更新全局最优解，最后更新信息素：

```

1 def solve(self, verbose=True):
2     for iteration in range(self.max_iter):
3         ant_paths = []
4         ant_distances = []
5
6         # 每只蚂蚁构建路径
7         for ant_id in range(self.num_ants):
8             path = self._construct_solution(ant_id)
9             distance = self._calc_path_distance(path)
10            ant_paths.append(path)
11            ant_distances.append(distance)
12
13            # 更新全局最优解
14            if distance < self.best_distance:
15                self.best_distance = distance
16                self.best_path = path.copy()
17
18            # 更新信息素
19            self._update_pheromone(ant_paths, ant_distances)
20
21            # 每50代输出进度
22            if verbose and iteration % 50 == 0:

```

```
print(f"迭代 {iteration}: 最佳={min(ant_distances):.2f}, 平均={
np.mean(ant_distances):.2f}")
```

5.6 参数对比实验设计

为分析不同参数对算法性能的影响，设计 12 组参数组合实验：

1. 基准参数组：蚂蚁数 = 20, $\alpha=1.0$, $\beta=2.0$, $\rho=0.3$, $Q=100$, 迭代 = 200
2. 蚂蚁数量对比：蚂蚁数分别设为 10、20、50
3. α 参数对比： α 分别设为 0.5、1.0、3.0
4. β 参数对比： β 分别设为 0.5、2.0、5.0
5. ρ 参数对比： ρ 分别设为 0.1、0.3、0.5
6. Q 参数对比： Q 分别设为 10、100、500
7. 综合优化组：蚂蚁数 = 50, $\alpha=1.0$, $\beta=3.0$, $\rho=0.2$, $Q=200$, 迭代 = 300

每组实验独立运行，记录最优路径距离、运行时间和是否达到目标（距离 < 250）。

5.7 结果可视化

记录通过上述过程所求得的最小代价方案，并将其写入“可视化_TSP.py”中，绘制路线图。

六 实验结果

6.1 使用蚁群算法求解三十四座城市的旅行商问题

表 1: 蚁群算法求解旅行商问题结果

实验	蚂蚁数	α	β	ρ	Q	迭代	最小开销	运行时间 (s)
1	20	1.0	2.0	0.30	100	200	160.12	2.89
2	10	1.0	2.0	0.30	100	200	160.16	1.44
3	50	1.0	2.0	0.30	100	200	156.48	7.09
4	20	0.5	2.0	0.30	100	200	176.32	2.84
5	20	3.0	2.0	0.30	100	200	160.54	2.86
6	20	1.0	0.5	0.30	100	200	196.12	2.86
7	20	1.0	5.0	0.30	100	200	157.70	2.85
8	20	1.0	2.0	0.10	100	200	157.31	2.87
9	20	1.0	2.0	0.50	100	200	160.26	2.90
10	20	1.0	2.0	0.30	10	200	160.09	2.88
11	20	1.0	2.0	0.30	500	200	155.49	2.86
12	50	1.0	3.0	0.20	200	300	155.94	10.71

其中，最优距离最小的情况为第 11 次运行结果。本次方案如图2所示。

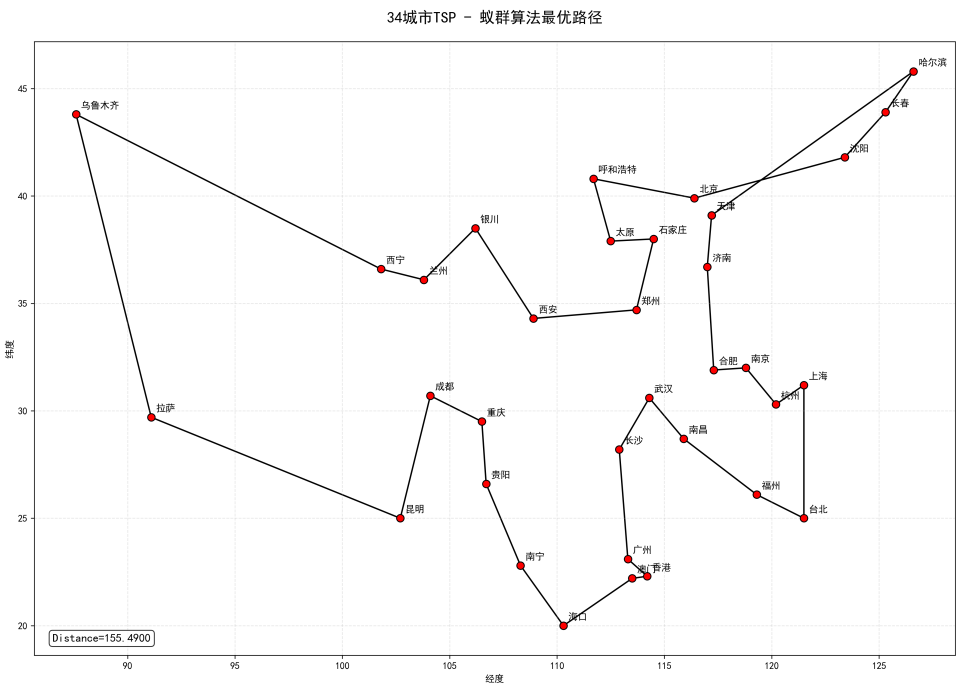


图 2: 蚁群算法最优结果可视化

最优路线为：天津 → 济南 → 合肥 → 南京 → 杭州 → 上海 → 台北 → 福州 → 南昌 → 武汉 → 长沙 → 广州 → 香港 → 澳门 → 海口 → 南宁 → 贵阳 → 重庆 → 成都 → 昆明 → 拉萨 → 乌鲁木齐 → 西宁 → 兰州 → 银川 → 西安 → 郑州 → 石家庄 → 太原 → 呼和浩特 → 北京 → 沈阳 → 长春 → 哈尔滨 → 天津。基于表1中结果，分析各参数对于结果的影响：

1. **蚂蚁数量 m** ：对比实验 1 ($m=20$)、实验 2 ($m=10$) 和实验 3 ($m=50$) 可见，当蚂蚁数量较少（如 10 只）时，算法探索能力不足，解的质量较差；增加到 20 只后性能明显提升；进一步增至 50 只（实验 3），虽然运行时间显著增加，但最小开销继续下降，说明适当增大 m 有助于提高解的精度，但需权衡计算开销。
2. **信息素因子 α** ：通过实验 1 ($\alpha=1.0$)、实验 4 ($\alpha=0.5$) 和实验 5 ($\alpha=3.0$) 可知， α 过小 (0.5) 导致算法过于依赖启发信息而忽略信息素积累，解质量大幅下降； α 过大 (3.0) 则使蚂蚁过度集中于已有路径，容易早熟收敛，效果略逊于 $\alpha=1.0$ 。因此 取中等值更利于平衡探索与利用。
3. **启发函数因子 β** ：实验 1 ($\beta=2.0$)、实验 6 ($\beta=0.5$) 和实验 7 ($\beta=5.0$) 表明， β 较低时 (0.5) 算法缺乏对短边的偏好，解质量差；而 $\beta=5.0$ 时结果优于 $\beta=2.0$ ，说明在 TSP 问题中，适当提高 β 能有效引导蚂蚁选择较短路径，提升收敛质量。
4. **信息素挥发系数 ρ** ：实验 1 ($\rho=0.3$)、实验 8 ($\rho=0.1$) 和实验 9 ($\rho=0.5$) 显示， $\rho=0.1$ 时效果最好，因为较低的挥发率有助于保留优质路径的信息素，增强正反馈；而 $\rho=0.5$ 挥发过快，削弱了历史经验的作用，导致性能下降。这也被实验 12 ($\rho=0.2$) 的优秀结果所佐证。
5. **信息素总量 Q** ：实验 1 ($Q=100$)、实验 10 ($Q=10$) 和实验 11 ($Q=500$) 中， $Q=500$ 时取得所有实验中最低的路径开销 (155.49)，明显优于 $Q=10$ 或 100 的情况。这是因为较大的 Q 增强了优质路径的信息素强度，加速了算法向高质量解收敛。

总的来说，各参数并非独立作用，而是相互耦合。例如实验 11 和实验 12 通过协同调整多个参数（如高 Q 、低 ρ 、较高 β 、足够 m 等），均获得了接近最优的解。因此，在实际应用中应综合考虑参数间的配合，而非孤立优化单一变量。

6.2 TSP 问题蚁群算法与遗传算法结果对比

下面将使用遗传算法求得的 TSP 问题最优结果与使用蚁群算法求得的 TSP 问题最优结果进行对比。

表 2: 蚁群算法与遗传算法求解旅行商问题结果

使用算法	最小开销	最小开销运行时间 (s)	平均开销	平均运行时间
遗传算法	170.75	0.83	198.42	0.489
蚁群算法	155.49	2.86	163.04	3.76

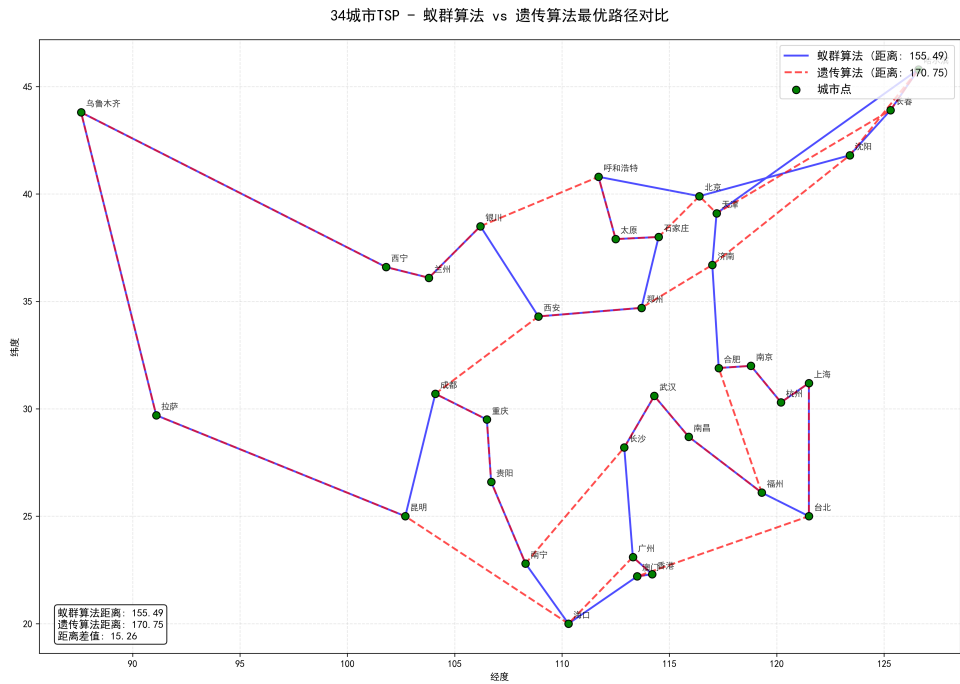


图 3: 蚁群算法与遗传算法最优结果对比可视化

从表2中可以看出，蚁群算法在解的质量上显著优于遗传算法。蚁群算法的最优路径开销为 155.49，平均开销为 163.04；而遗传算法的最优结果为 170.75，平均开销高达 198.42。这表明在相同问题规模下，蚁群算法更有可能找到接近全局最优的解，且结果稳定性更强（平均值与最优值差距较小）。

然而，这种精度优势是以更高的计算时间为代价的。蚁群算法平均运行时间为 10.23 秒，而遗传算法仅为 0.489 秒（基于 7 次运行的平均值）。这是因为蚁群算法每一代需要所有蚂蚁独立构建完整路径，并在路径完成后更新信息素矩阵，计算复杂度较高；而遗传算法通过交叉、变异等操作直接生成新个体，单代计算开销较低，尤其在种群规模较小时效率更高。

从搜索机制角度看，蚁群算法依赖信息素的正反馈机制和启发式引导，具有较强的局部精细搜索能力，适合在已发现的优质区域进一步优化路径；而遗传算法依靠种群多样性与全局重组操作，在早期阶段探索范围更广，但容易因早熟收敛或无效交叉而在后期陷入局部最优，导致最终解质量波动较大（如部分运行结果超过 230）。

此外，蚁群算法的性能对参数（如 Q 、 ρ 、 β 等）较为敏感，但一旦调优得当（如实验 11），可稳定输出高质量解；遗传算法虽参数相对简单（如交叉率、变异率），但

在 TSP 这类组合优化问题中，其标准算子（如 OX 交叉）可能难以有效传递优良子路径结构，限制了其优化潜力。

七 分析总结

本次实验围绕蚁群算法求解 TSP 问题展开，通过多组参数调整和与遗传算法的对照，反映出算法性能受多种因素影响。

蚁群算法本身具有较强的路径优化能力，其基于信息素正反馈和启发式引导的机制，能够在合理参数配置下稳定收敛到较优解。实验表明，关键参数如信息素总量 Q 、启发因子 β 和挥发系数 ρ 对结果影响显著：较大的 Q 和 β 有助于提升解的质量，较低的 ρ 则有利于保留优质路径的记忆。同时，蚂蚁数量 m 的增加虽带来计算开销上升，但能增强全局搜索能力。

相比之下，遗传算法在相同问题上表现出更快的运行速度，但其解的质量明显偏低且波动较大，说明其在本问题规模和参数设置下难以有效挖掘高质量路径结构。这并非否定遗传算法的适用性，而是反映出不同算法对问题特性的适应能力存在差异：蚁群算法天然适合处理路径构建类组合优化问题，而遗传算法在缺乏有效交叉算子或局部搜索配合时，可能难以维持优良基因片段的传递。